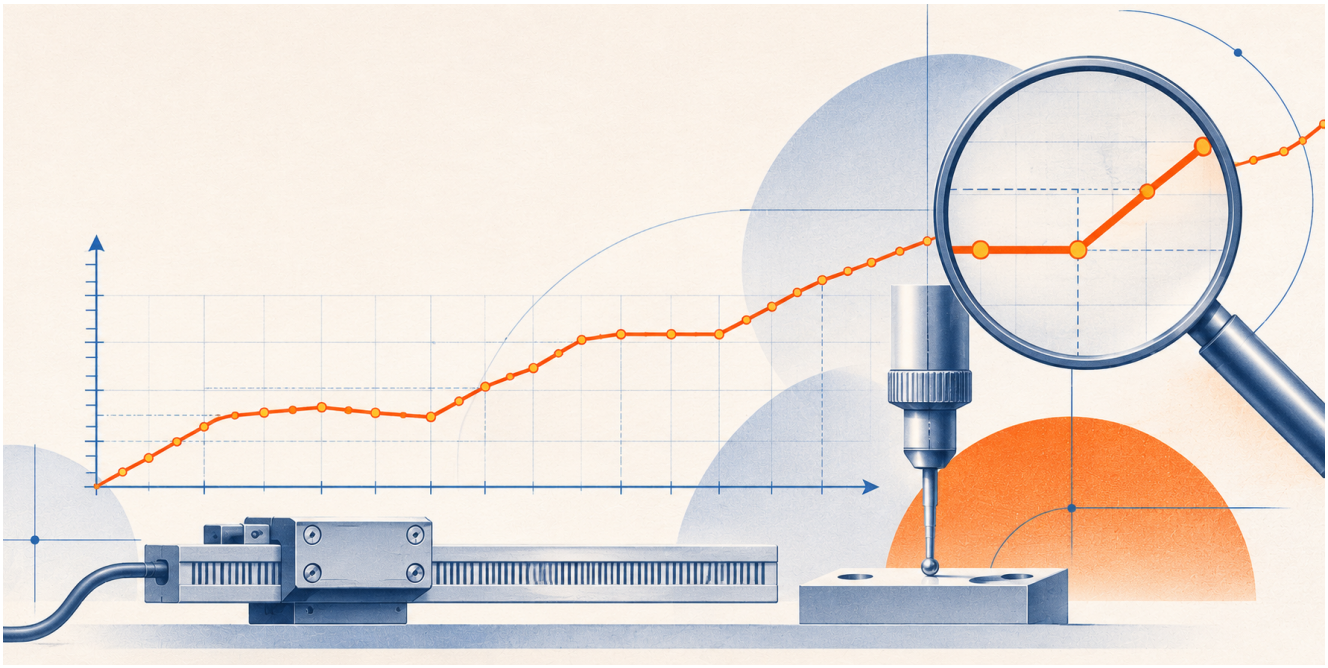
paceval. | SCIENTIFIC FUNCTIONALITY STUDY

Industrial Calibration at Formula Scale

Long Piecewise Mathematical Functions, Versioned Execution Graphs and Auditable Mathematical Quality Margins



13 November 2026 | English research report | Real paceval SDK 4.25 execution

Scientific claim. This demonstrator establishes mathematical feasibility: a compatible paceval Linux runtime created and executed 121 persistent graphs for two 4,096-interval calibration models, and 569 graphs for two 65,536-interval models. Each execution was checked independently. No real measurement traceability, product certification, instrument accuracy or hardware acceleration is claimed.

The title image is an editorial scientific illustration, not a photograph of tested laboratory equipment.

01 Research question and industrial scope

Can an industrial host use one versioned mathematical execution layer to evaluate large sensor transfer functions, temperature and drift corrections, analytical sensitivities, uncertainty expressions and auditable quality-decision margins without rewriting its supervisory code when a calibration formula changes?

The predecessor study already established a longer than seven-million-character piecewise formula and modular 65,536-segment execution. This expanded study concentrates on a complete functional chain, preserving two versions simultaneously and checking both over synthetic sensor inputs.

The chosen first application is read-only, non-safety-critical machine-axis and precision-sensor correction. Control actuation, true device calibration and decisions with safety consequences are excluded.

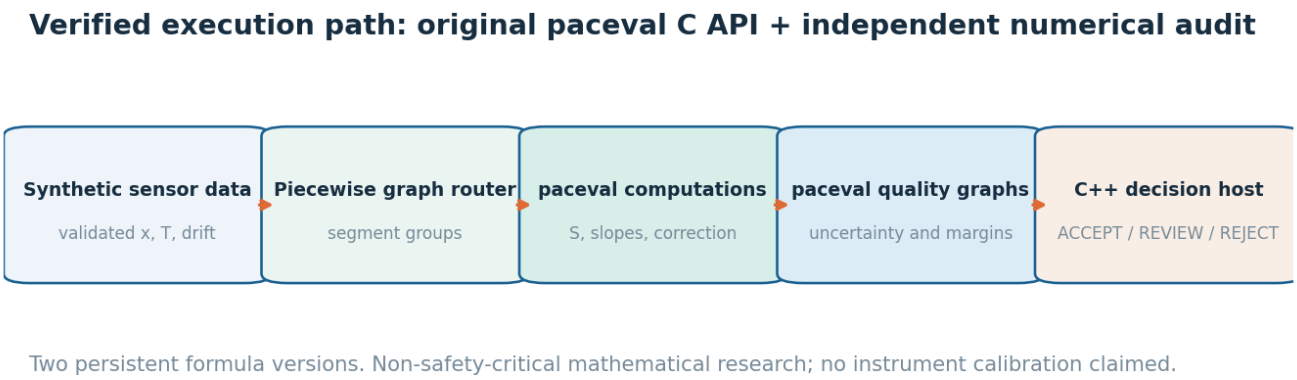


Figure 1. Verified execution responsibilities. Formula computation and mathematical guard margins are inside paceval. The C++ program handles input validation, routing, independent reference checks and status selection.

The literature distinguishes mathematical calibration algorithms from metrological traceability. NIST requires a documented calibration chain and associated uncertainty; IEC 61298-2:2026 covers designated reference-condition performance testing but excludes process measurement transmitters [2, 3].

02 Model definition and endpoint ownership

The original three-point example is preserved as a real paceval scalar graph. It returns the expected values 6, 4.5, 3, 14 and 25 at inputs 1.5, 2.0, 2.5, 3.0 and 3.5 respectively. Its generalization uses a uniform piecewise-linear interpolation model for x in [0, 1].

$$S_v(x) = y_{i,v} + \frac{y_{i+1,v} - y_{i,v}}{x_{i+1} - x_i}(x - x_i)$$

The data knots are synthetic samples of an explicitly stated oscillatory function. Model v2 modifies model v1 by 0.002 sin(5x) + 0.001x. For all but the final interval, each knot belongs to the segment on its right; the final interval owns x = 1. Out-of-range and nonfinite inputs are rejected before graph evaluation.

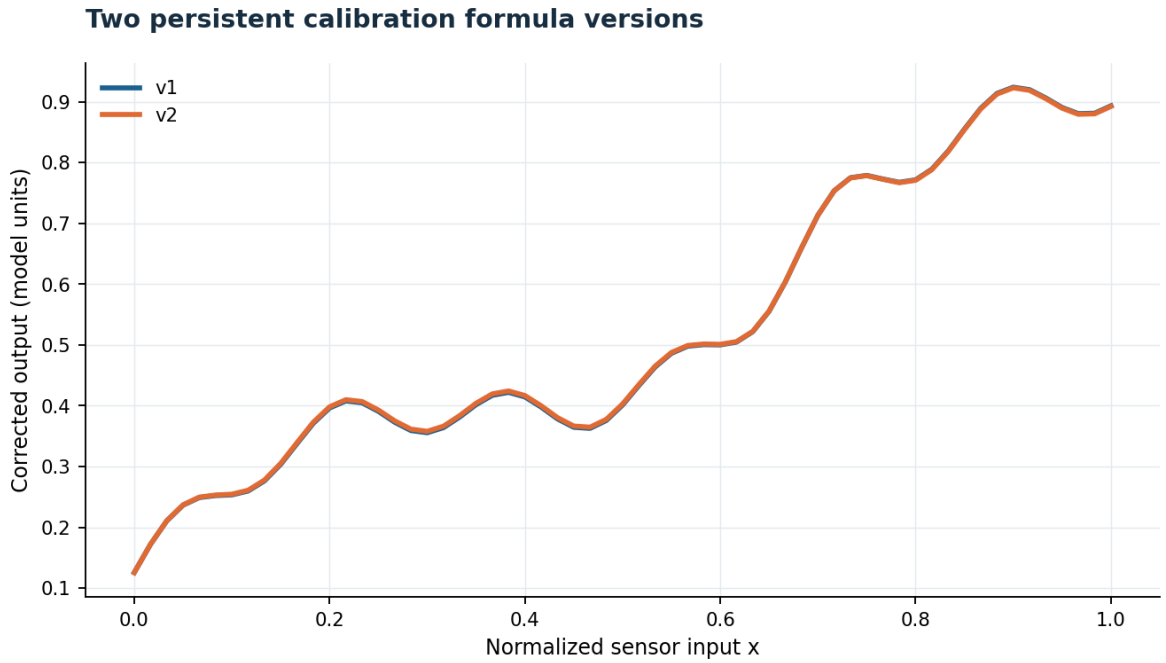


Figure 2. The two models can be created, retained and executed independently through persistent paceval handles; both curves are shown at T = 30 °C.

Model revision here means a controlled mathematical parameter change, not recalibration of physical equipment.

03 Temperature and drift compensation

Industrial calibration often involves operating conditions besides the raw sensor reading. The demonstrator composes the interpolated value with explicitly synthetic temperature and drift corrections, without replacing the surrounding control and measurement infrastructure.

$$G_v = S_v(x)[1 + 0.00035(T - 20)] + 0.0005(T - 20)$$

$$+ 0.000075\sin(7x)(T - 20) + d(0.001 + 0.0015x)$$

This is a version-independent formula graph that receives the version-specific interpolant S and local slope from the routed paceval segment graphs. The input domain is 0 to 60 °C and normalized drift from -1 to 1. All numeric coefficients are illustrative and dimensionless.

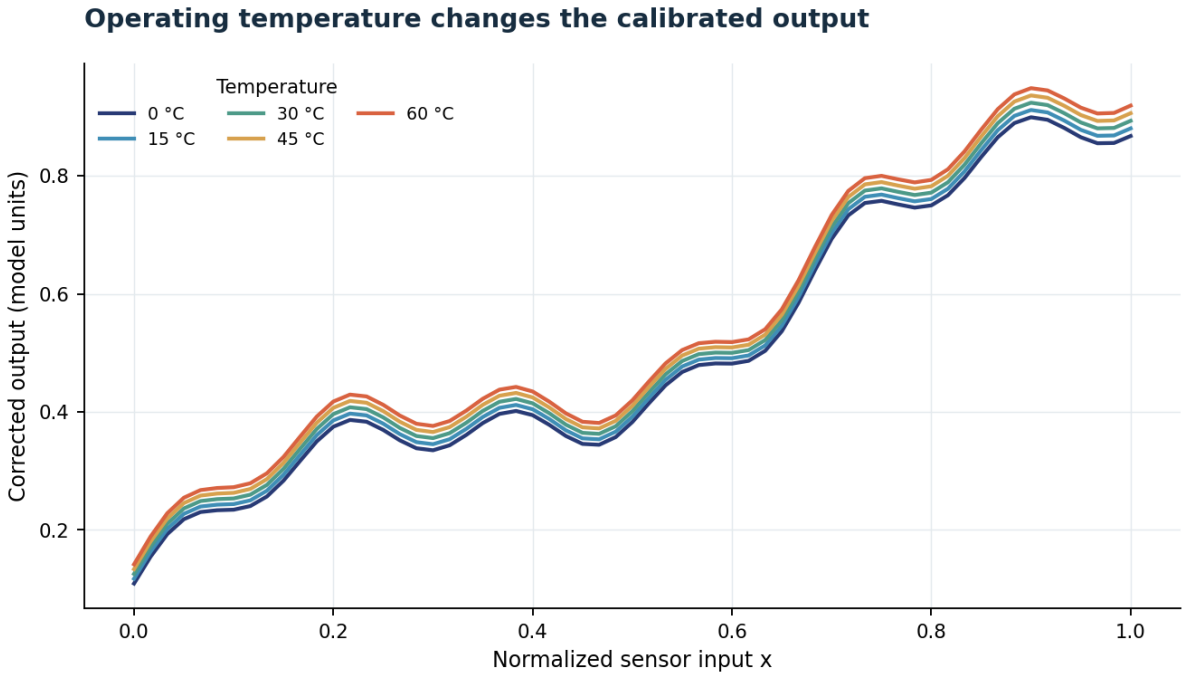


Figure 3. Real paceval evaluations of the corrected v1 output at different synthetic operating temperatures. The distinct curves demonstrate a combined reusable mathematical expression.

04 Full two-variable response surface

The graph registry contains the precise text of every executed formula. For scientific visualization, the C++ demonstrator evaluated the actual paceval functions on a declared 61 by 41 grid of raw signal and temperature values. No separate Python implementation was substituted to generate the plotted surface.

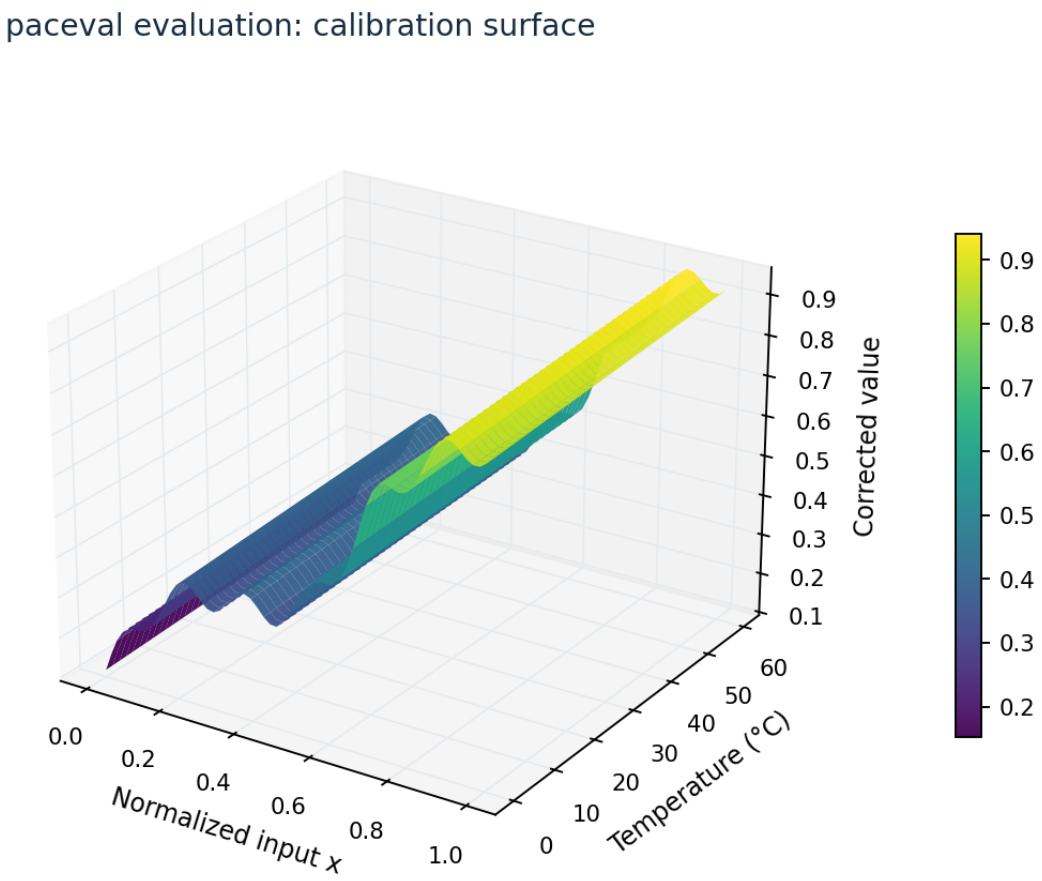


Figure 4. Corrected response surface from the original paceval SDK, model v1, normalized drift d = 0. The surface represents a constructed mathematical calibration model, not sensor measurements.

For inspection and reproduction, the underlying data are stored in results/calibration_surface.csv. Identical equations and parameter values are used for the separate reference audit.

05 Versioning, concurrent execution and rollback readiness

Both formula sets are active simultaneously. Each graph has a version identifier, role, mathematical expression, declared input schema and unit description. A version comparison is recorded for all 720 synthetic input samples. This demonstrates how a supervisory application can preserve earlier mathematical behavior while evaluating a revision.

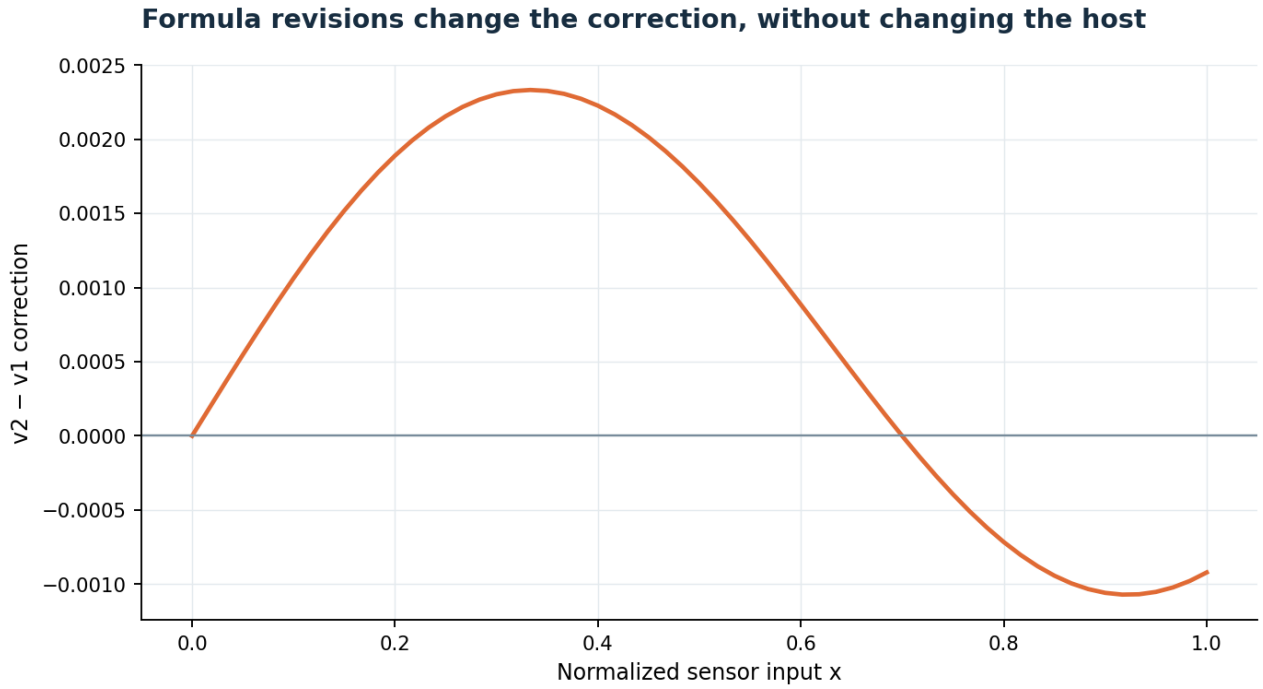


Figure 5. Differences between independently computed v2 and v1 corrected values at T = 30 °C. Neither curve is presented as a more accurate model.

A production lifecycle would additionally require signed calibration artifacts, authorized approval, provenance, expiration policies, controlled activation, rollback and independent metrological requalification. The demonstrator does not claim to implement these operational mechanisms.

06 Synthetic residuals and targeted anomaly scenarios

Four deterministic synthetic groups exercise different aspects of the formula execution: machine-axis correction, precision sensor, quality gate and temperature cycle. The quality-gate inputs contain deliberate artificial offsets to exercise all three decision states. There is no real sensor defect dataset.

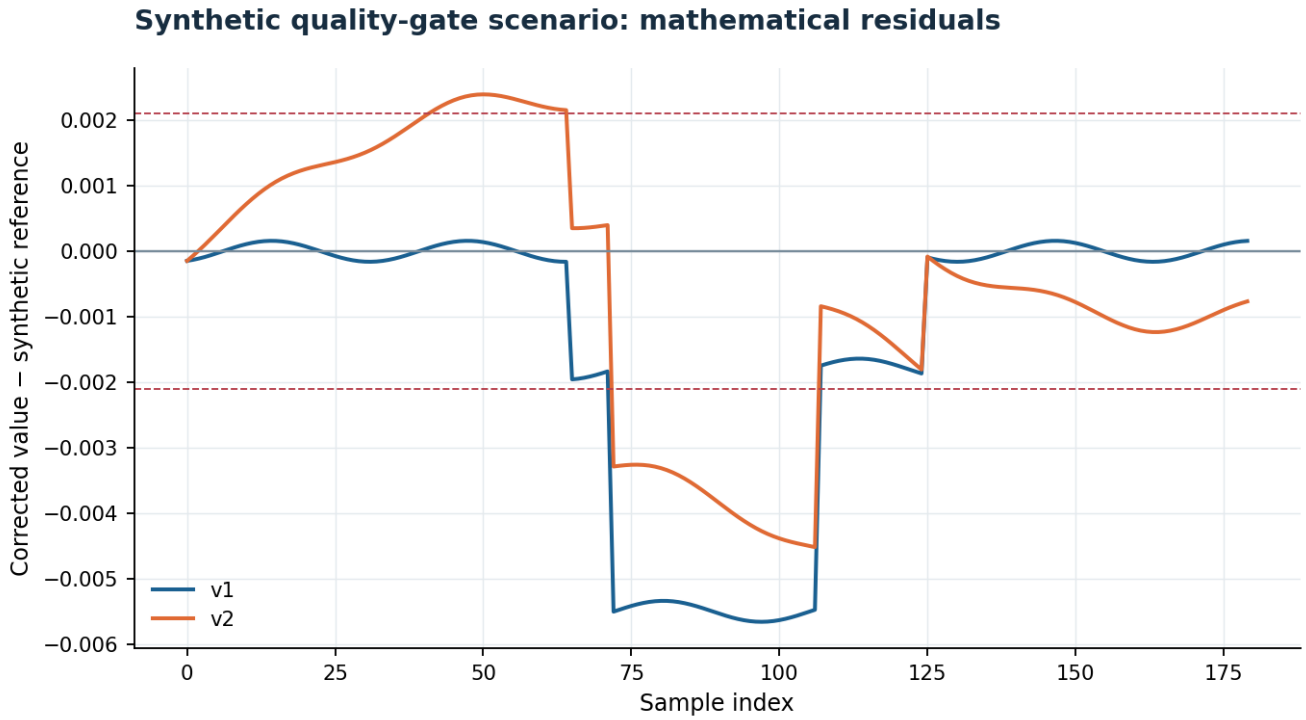


Figure 6. Residuals from real paceval evaluation for the synthetic quality-gate scenario. Dashed lines show the illustrative raw tolerance; final decisions use uncertainty-aware margins.

The reference signal originates from the source synthetic generating function and is deliberately perturbed by known artificial deviations. Thus these residuals are reproducible experiment inputs and not empirical measurement errors.

07 First-order propagation of uncertainty

Three analytical sensitivity functions are represented as paceval mathematical graphs, along with the combined standard uncertainty below. Input uncertainty terms are assumed uncorrelated in this first research model. A real measurement chain requires a justified covariance structure and a validated model uncertainty budget.

$$u_G = \sqrt{(G_x u_x)^2 + (G_T u_T)^2 + (G_d u_d)^2 + u_{model}^2}$$

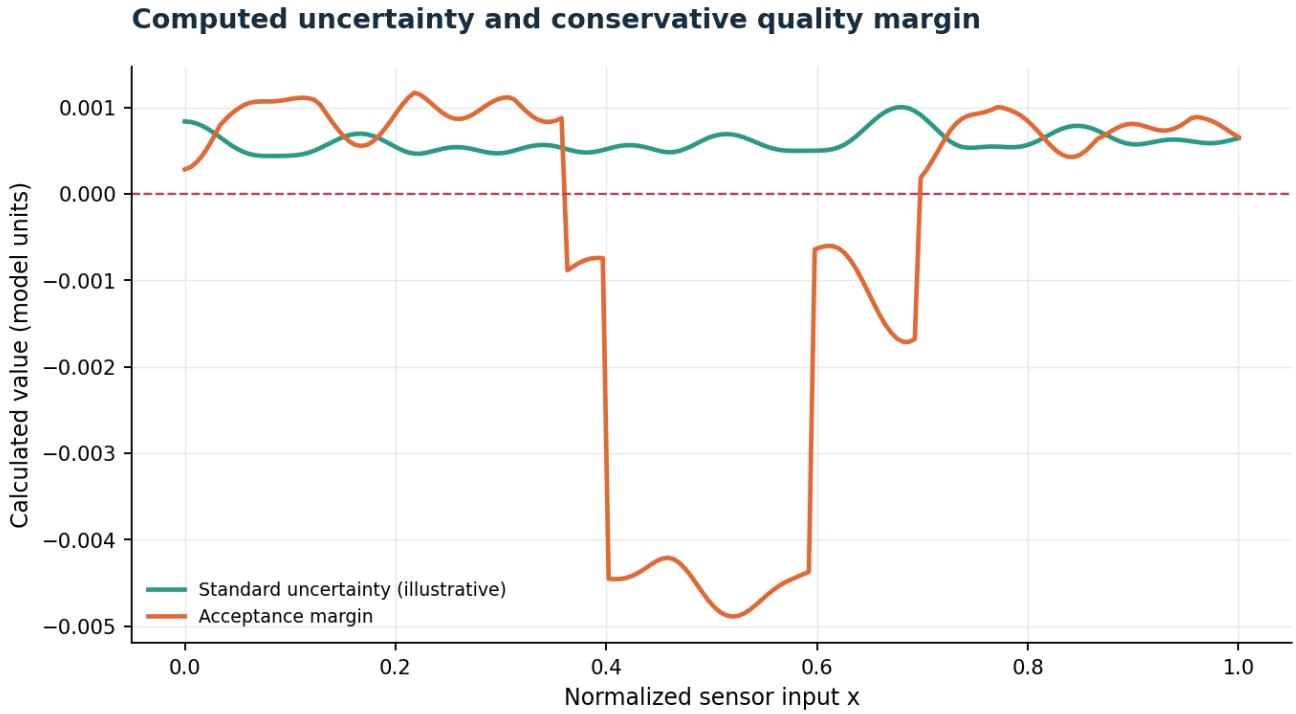


Figure 7. Version v1: the calculated standard-uncertainty quantity and acceptance margin are separate real paceval outputs for the synthetic quality-gate scenario.

These numbers do not establish coverage probability or traceable uncertainty. For non-smooth transfer functions, the local derivative at a knot is defined one-sided and is not a global differentiability certificate. This experiment does not enable TINC.

08 Analytical sensitivities and piecewise boundaries

A piecewise-linear graph has an ordinary derivative inside each open segment, but not generally at the internal knots. This study declares a right-segment sensitivity convention for internal knots and a last-segment convention at x = 1. It is used consistently by the native reference and independent 80-digit audit.

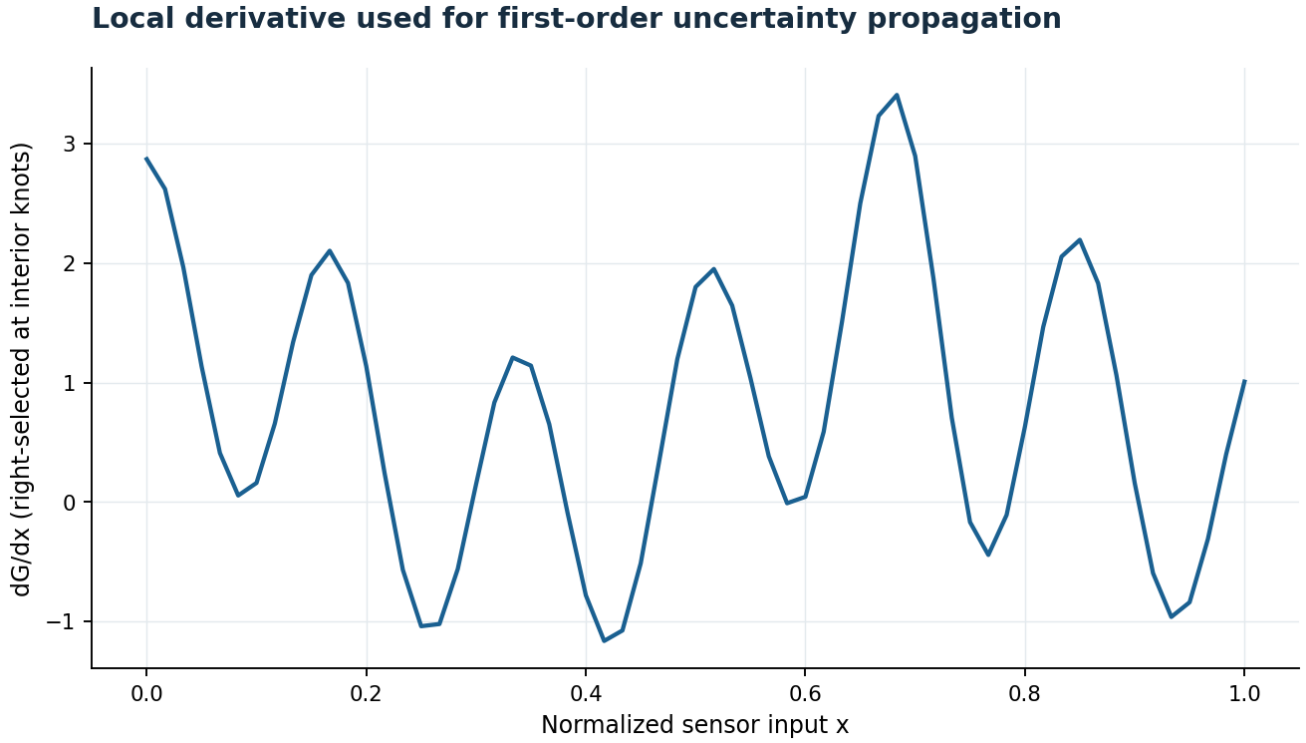


Figure 8. Local dG/dx at T = 30 °C. Visible changes follow the synthetic transfer-function structure and are computed from persistent paceval derivative graphs.

For instruments requiring continuous first derivatives, a shape-preserving cubic Hermite interpolation may be a later model choice; SciPy documents PCHIP as shape-preserving and continuously first differentiable [5]. PCHIP is not implemented in this demonstrator.

09 Mathematical quality decisions

The host does not invent a numeric safety decision. paceval evaluates residuals, the combined uncertainty and two explicit guard margins, after which the host performs only the three-way comparison.

$$m_{accept} = t - |G - r| - 2u_G \quad ; \quad m_{reject} = t - |G - r| + 2u_G$$

ACCEPT if m_accept ≥ 0; REJECT if m_reject < 0; otherwise REVIEW. The multiplier 2 is an arbitrary research policy parameter, not evidence of a metrological or statistical coverage guarantee.

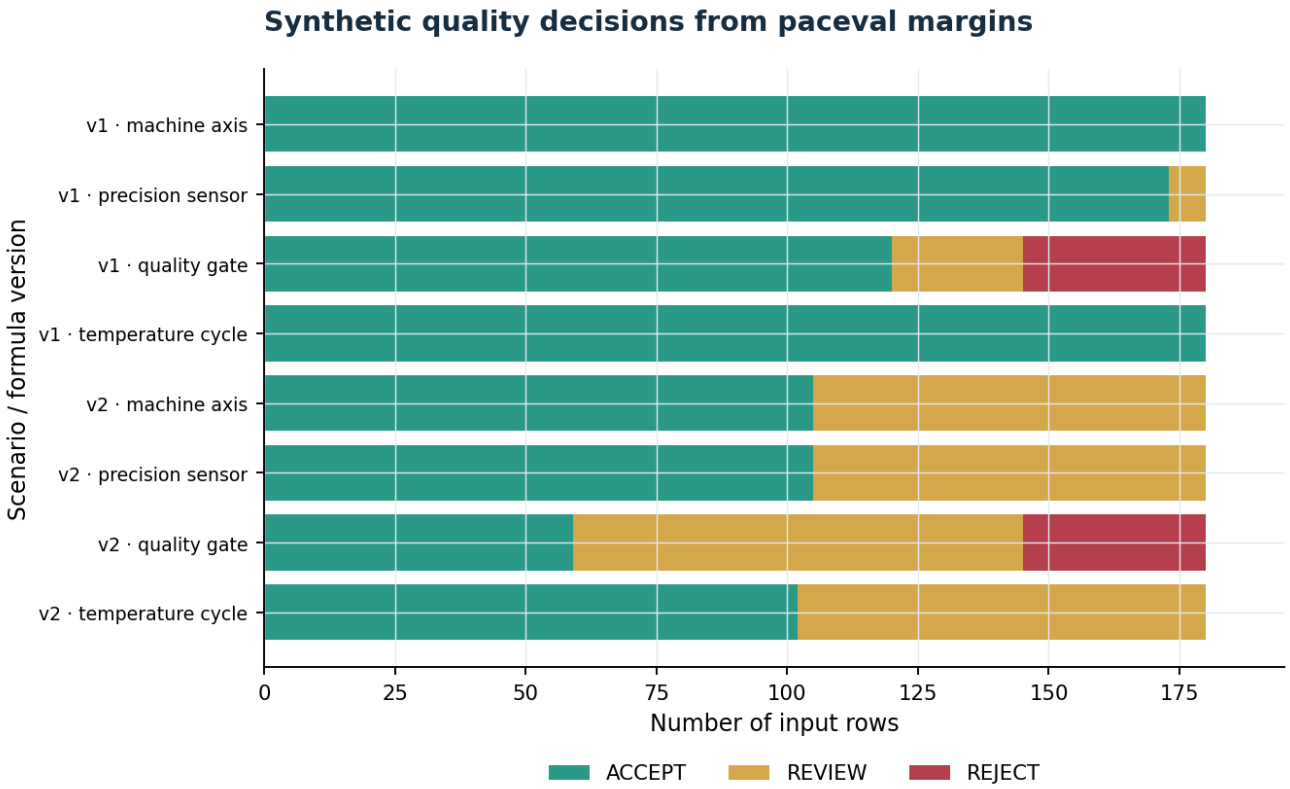


Figure 9. Outcomes of 720 deterministic inputs evaluated independently in each model version. Each status is selected from actual paceval mathematical margins.

10 Governance score family

Beyond corrected measurements, the demonstrator constructs 16 different persistent composite score expressions per model version. Each score combines absolute residual, calculated uncertainty, local derivative sensitivity and operating drift using explicit versioned coefficients.

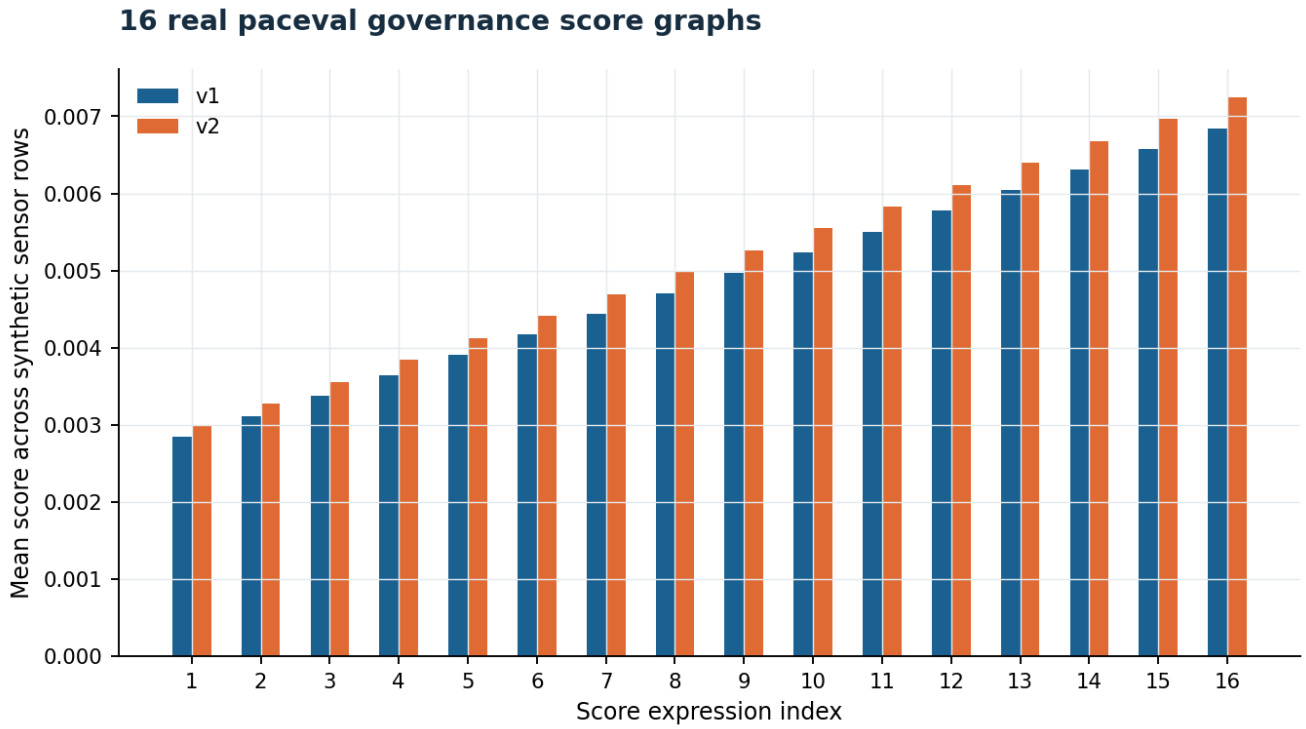


Figure 10. Sixteen distinct graph expressions are evaluated for every synthetic input and version. These score values are not probabilities or approved industrial quality metrics.

The score family shows how formula-governance requirements may be tested as mathematics without changing the C++ data acquisition or scheduling host. All generated strings are retained in results/graph_registry.csv.

11 Large expression capacity and routed batch execution

The original historical report demonstrated a single 7,712,198-character monolithic function with 65,536 segments. Here we independently re-executed the full two-model framework at 65,536 segments per version, with 512 masked segments per graph, using persistent graph handles and routed single-graph batches.

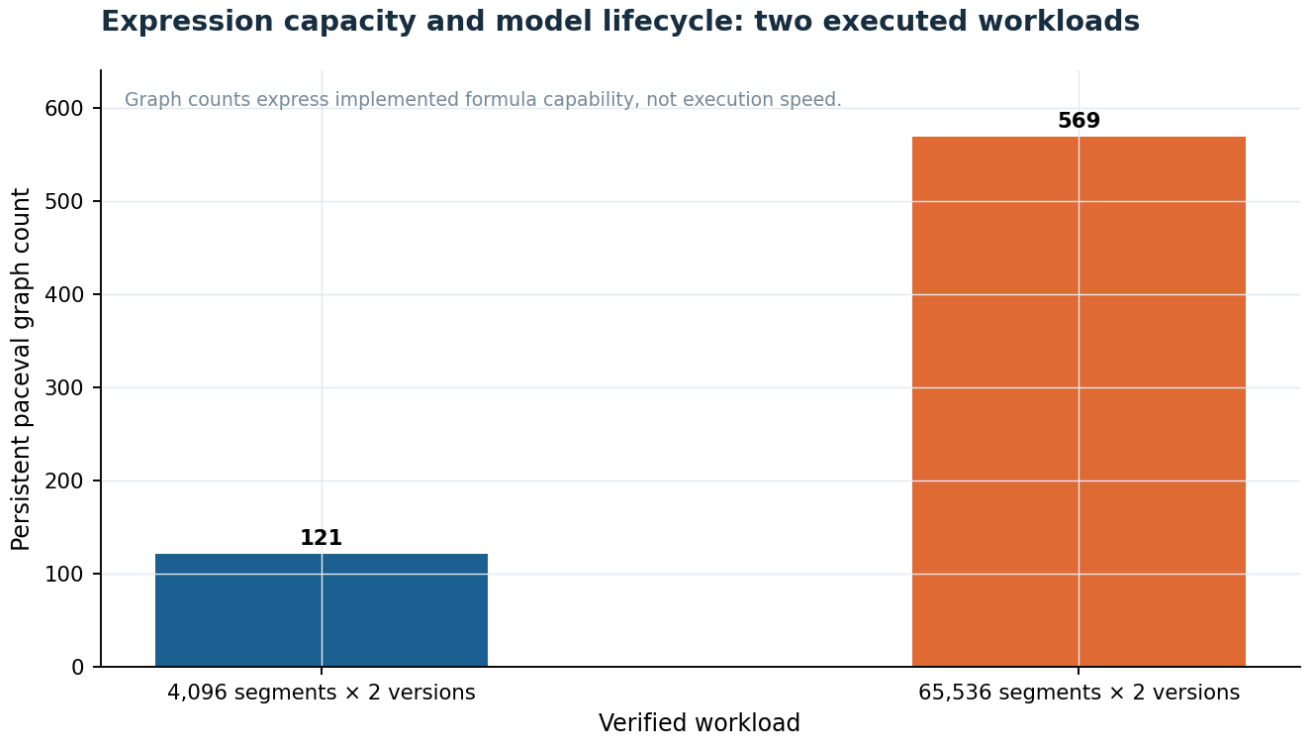


Figure 11. 121 persistent computation objects for the primary 4,096-interval case and 569 for the dual-model 65,536-interval capacity case. This is a count of executed mathematical functionality, not a performance comparison.

The 569-graph capacity run includes 512 interpolation and slope graphs, 56 postprocessing graphs, and one original three-point historical anchor. The independent 80-digit audit succeeded separately for this capacity run.

12 Acceptance-margin surface and operating context

In addition to the calibrated response surface, a second actual paceval output grid shows how a synthetic tolerance policy interacts with a revised formula. This illustrates a mathematically explicit decision surface, not a validated threshold policy for any production machine.

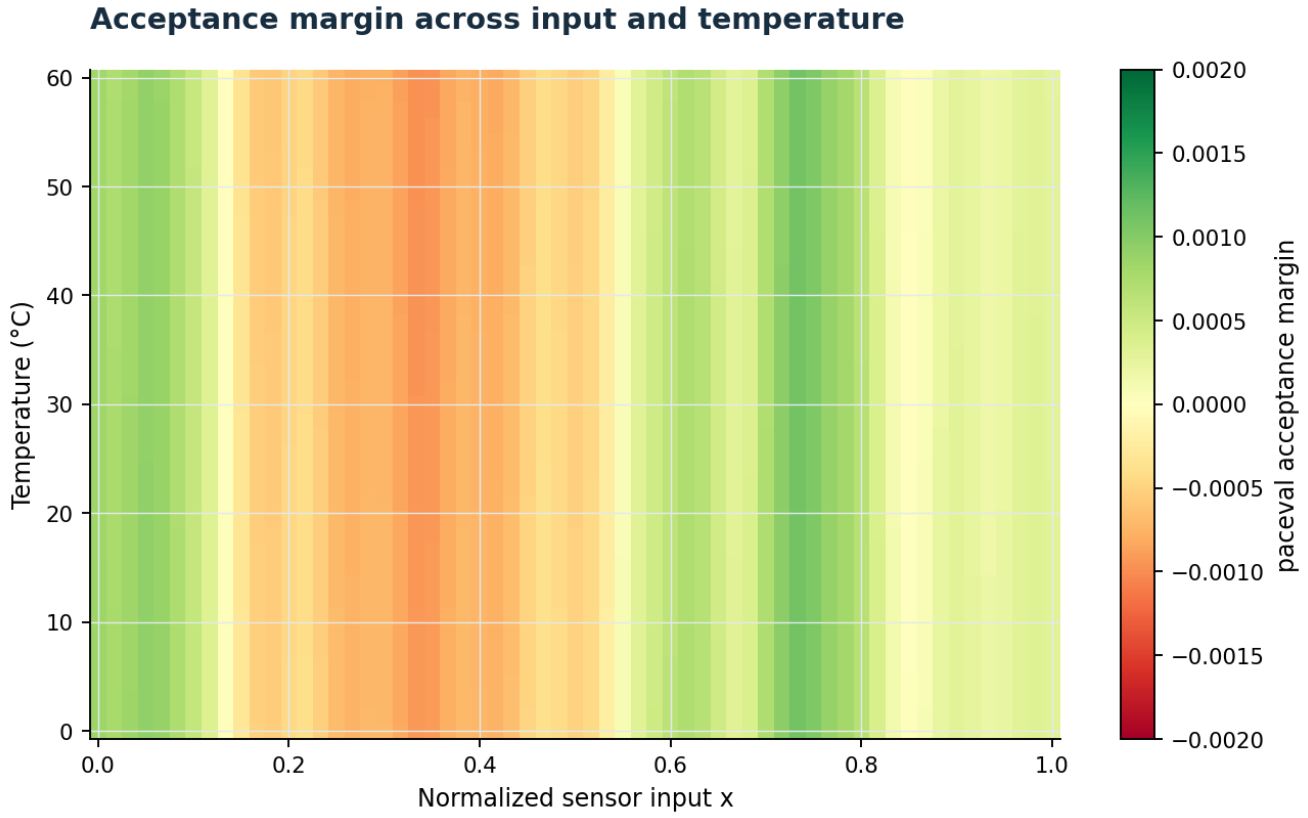


Figure 12. Version v2 acceptance margin across normalized sensor input and temperature. The recorded paceval graph outputs can be recalculated by rebuilding and executing the project.

An actual industrial implementation must agree on a qualified limit, a documented uncertainty model and explicit behavior near the boundary. Observed agreement on a finite sample is not a general proof of margin stability.

13 Validation and reproducibility

All numerical claims below refer to completed executable runs with the original paceval SDK 4.25 shared library, and not to assumed outcomes. The C++ source includes a separately implemented mathematical reference used for every stored input; an independent Python-mpmath script recomputes the 11 quantities with 80-decimal-digit arithmetic.

Check	4,096 intervals × 2	65,536 intervals × 2
Executed persistent paceval graphs	121	569
Synthetic input rows	720	720
Actual paceval graph output values	193,265	193,265
Independent native quantity assertions	15,840	15,840
80-decimal-digit numerical checks	15,840	15,840
Largest high-precision absolute difference	1.07e-12	1.42e-11
Decision agreement with decimal oracle	PASS	PASS

Numerical differences are in normalized synthetic model units. The high-precision check uses an independently written mathematical formula and high-precision transcendentals; its small disagreement with binary64 partly reflects coefficient rounding, not a measurement-accuracy result.

The source uses C++17 and the original paceval API. Batch routing is the C++ host responsibility; interpolation, correction, derivative, uncertainty and mathematical scoring are real paceval expressions. Test commands are documented in README.md. The Linux runtime library remains external to the publication.

Reproduction assets: input manifest, exact formula registry, program sources, recorded graph outputs, 80-digit audit JSON, capacity-run evidence and 12 regenerable scientific figures. No timing or acceleration comparisons are part of this functionality study.

14 Limitations, standards, and next experimental milestone

This is a verified mathematical software demonstrator, not accredited instrument calibration. All knot values, reference data, drift factors, quality tolerances and standard uncertainties are synthetic. It contains no traceable calibration chain, real sampling hardware, physical uncertainty budget, industrial validation, safety certification or certified interval-enclosure evidence.

Relevant external context: NIST distinguishes measurement-result traceability from software calculations [2]. IEC 61298-2:2026 applies to process-instrument tests under reference conditions and excludes process measurement transmitters regulated in IEC 62828 [3]. ISO/IEC 17025 specifies laboratory competence requirements and does not certify this project by reference alone [4].

The next justified applied pilot is a sensor or machine axis with an approved measurement dataset, reference equipment and its full uncertainty statement. Formula updates and approval/rollback workflows should be independently audited against an instrument-qualified model. OPC UA companion information models are one possible future integration interface [6].

15 References

[1] paceval. Mathematical Engine C/C++ API version 4.25. https://paceval.com/api/

[2] NIST. Metrological Traceability: Frequently Asked Questions and NIST Policy.
https://www.nist.gov/metrology/metrological-traceability

[3] IEC 61298-2:2026. Process measurement and control devices, reference-condition performance tests.
https://webstore.iec.ch/en/publication/87417

[4] ISO/IEC 17025:2017. General requirements for the competence of testing and calibration laboratories.
https://www.iso.org/standard/66912.html

[5] SciPy PchipInterpolator reference and associated Fritsch/Butland method.
https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.PchipInterpolator.html

[6] OPC Foundation, OPC UA Companion Specifications.
https://opcfoundation.org/about/opc-technologies/opc-ua/ua-companion-specifications/

[7] NIST Technical Note 1297, Guidelines for Evaluating and Expressing the Uncertainty of NIST Measurement Results.
https://www.nist.gov/pml/nist-technical-note-1297

Research conclusion. Versioned, long piecewise calibration and uncertainty-aware mathematical scoring are demonstrably executable by real paceval graphs. Practical sensor calibration benefits require a separate measurement-based study.